

APP, @MAIN & SCENES

A SwiftUI app is a type conforming to `App`, marked `@main`, with `body: some Scene`. Scene types: `WindowGroup`, `Window`, `UtilityWindow`, `DocumentGroup`, `Settings`, `MenuBarExtra`, `ImmersiveSpace`. `@Environment(\.scenePhase)` reads the current `ScenePhase`. UIKit/AppKit bridges: `UIApplicationDelegateAdaptor`, `NSApplicationDelegateAdaptor`. State restoration uses `@SceneStorage`.

#PREVIEW & XCODE PROJECT

Declare a preview with the `#Preview` macro; runs on the main actor. `PreviewProvider` is deprecated—convert to `#Preview`.

```
#Preview("Name", traits:
.sizeThatFitsLayout) { ... }; also
.landscapeLeft,
.fixedLayout(width:height:), and
arguments: for multiple samples. @Previewable
lets a preview declare @State inline. #Playground
is the non-UI counterpart.
```

VIEW + MODIFIER ORDER

A view is a description, not an object: `body: some View`. Modifier order matters—each modifier wraps the view in a new one, so e.g. `padding` then `background` differs from `background` then `padding`.

Build a custom `ViewModifier` for reusable chains. `ViewBuilder` / `ContentBuilder` compose conditional content. Structural vs. explicit view identity (`id` modifier) affects state lifetime. Custom environment values via the `@Entry` macro.

LAYOUT

SwiftUI layout: propose, choose, place. Stacks: `HStack`, `VStack`, `ZStack`, plus lazy `LazyVStack`/`LazyHStack`. Grids: `Grid`/`GridRow` (fixed), `LazyVGrid`/`LazyHGrid` with `GridItem` (scrolling).

`ViewThatFits` picks the first alternative whose ideal size fits. Custom arrangements via the `Layout` protocol, switchable at runtime with `AnyLayout`. Also: `frame`, `padding`, `Spacer`, alignment guides, `containerRelativeFrame`, `onGeometryChange`, `GeometryReader`, size classes.

STATE WRAPPERS

Wrapper	Use
<code>@State</code>	Storage a view owns, tied to its identity (built on the <code>State()</code> macro in Xcode 27)
<code>@Binding</code>	Read-write reference to someone else's source of truth (e.g. <code>\$selection</code>)
<code>@Observable</code>	Marks a reference-type model; SwiftUI tracks which properties a body reads
<code>@Bindable</code>	Local variable/property giving Bindings into an <code>@Observable</code> object's fields
<code>@Environment</code>	Reads a value/model injected via <code>.environment(store)</code> , e.g. <code>@Environment(NoteStore.self)</code>
<code>@AppStorage</code>	Backed by <code>UserDefaults</code> ; shared by every view declaring the same key
<code>@SceneStorage</code>	Backed by the scene's restoration archive

LISTS / FOREACH / TABLE / SEARCHABLE

`List` + `ForEach` over `Identifiable` data, with `Section` for grouping. `onDelete(perform:)` / `onMove(perform:)` on a `ForEach` enable edit mode. `swipeActions` works on any container (new in 27); `reorderable()` + `reorderContainer` enable drag-to-reorder.

`Table(rows, selection:, sortOrder:)` with `TableColumn("Title", value: \.title)` on macOS/iPadOS; supports static rows and hierarchical/disclosure rows. `searchable` adds a search field; `refreshable` adds pull-to-refresh.

NAVIGATION & PRESENTATION

`NavigationSplitView` (multi-column), `NavigationStack` with value-based `navigationDestination`. Programmatic navigation via `NavigationPath`, a type-erased list of hashable values; its codable property (`NavigationPath.CodableRepresentation`) can be saved to `@SceneStorage`.

`TabView` built from `Tab` values (not `tabItem`); `.tabViewStyle(.sidebarAdaptable)`; `TabRole.prominent` (new in 27) emphasizes one tab.

```
.sheet(isPresented:) with
.presentationDetents([.medium, .large]);
.inspector(isPresented:);
.alert(error:actions:);
.confirmationDialog(_:item:titleVisibilit
y:actions:).
```

CONTROLS & FORMS

Form for settings-style screens. Value controls: `Toggle`, `Picker`, `Slider`, `Stepper`, `DatePicker`. `Menu` and `LabeledContent`. Focus via `@FocusState`.

`Text`, `AttributedString` and `Markdown`; `TextField` / `TextEditor` with the `.bordered` style. `Image` and `AsyncImage` (with its own cache, new in 27). SF Symbols: rendering modes, variable values, symbol effects. Dynamic Type support throughout.

ANIMATION & GESTURES

Implicit: `.animation(.snappy, value:)`. Explicit: `withAnimation(.bouncy) { ... }` runs a closure inside a transaction that carries the animation. `Animation.default` is a spring. Transitions, `matchedGeometryEffect`, multi-step `PhaseAnimator`, timeline `KeyframeAnimator`.

Gestures are values: `TapGesture`, `LongPressGesture`, `DragGesture`, `MagnifyGesture`, `RotateGesture`. `@GestureState` holds transient state that exists only while a gesture is active.

LIQUID GLASS

`glassEffect(_:in:)` renders Liquid Glass behind a view's content, clipped to a shape, e.g. `.glassEffect(.regular.tint(.accentColor).interactive(), in: .circle)`. Not available on visionOS.

`GlassEffectContainer` renders every `glassEffect` inside it in one pass; `glassEffectID(_:in:)` lets glass shapes morph across appear/disappear. `glassEffectUnion(id:namespace:)` merges several views into one glass shape. Glass button styles: `.buttonStyle(.glass)`, `.buttonStyle(.glassProminent)`, `.buttonStyle(.glass(.clear))`.

ACCESSIBILITY

Four VoiceOver properties: `accessibilityLabel`, `accessibilityValue`, `accessibilityHint`, and traits. `accessibilityElement(children: .combine)` merges a group into one element; `.contain` groups without merging; `.ignore` hides children.

`accessibilityRepresentation` replaces an element's accessibility content. `accessibilityAction(named:)` adds custom actions; adjustable controls use an adjustable action. Respect Dynamic Type, Reduce Motion, Reduce Transparency, Increase Contrast, Differentiate Without Color.

LOCALIZATION

String Catalogs (`.xcstrings`, e.g. `Localizable.xcstrings`) hold keys, comments, plural and device variations, generated by the compiler from source. Three text types: `LocalizedStringKey` (SwiftUI's default), `LocalizedStringResource` (deferred lookup), and `String(localized:comment:)` (immediate lookup, returns a `String` now).

Generated localizable symbols, automatic grammar agreement/inflection, `FormatStyle` for dates/numbers/lists/measurements, right-to-left layouts, XLIFF/.xcloc exchange.

UIKIT / APPKIT INTEROP

`UIViewRepresentable` / `UIViewControllerRepresentable` and `NSViewRepresentable` wrap UIKit/AppKit views for use inside SwiftUI, using a `Coordinator`.

`UINavigationController` is a `UIViewController` whose view is a SwiftUI hierarchy—the main door from UIKit into SwiftUI, e.g. `UINavigationController(rootView: NoteDetail(note: note).environment(store))`.

`UIHostingConfiguration` hosts SwiftUI inside collection/table view cells. AppKit counterparts: `NSHostingController`, `NSHostingView`, `NSHostingMenu`.

SWIFT CHARTS

Swift Charts is Apple's declarative charting framework: a `Chart` builds a graph from small, composable marks (`BarMark`, `LineMark`, `PointMark` and others) instead of a `Canvas` drawing loop, and inherits SwiftUI's animation system.

Example: `Chart(dailyCounts) { entry in BarMark(x: .value("Day", entry.day, unit: .day), ...) }`.